A FAST METHOD FOR INFERRING HIGH-QUALITY SIMPLY-CONNECTED SUPERPIXELS

Oren Freifeld, Yixin Li, and John W. Fisher III

Massachusetts Institute of Technology
Computer Science and Artificial Intelligence Laboratory

ABSTRACT

Superpixel segmentation is a key step in many image processing and vision tasks. Our recently-proposed connectivity-constrained probabilistic model [1] yields high-quality superpixels. Seemingly, however, connectivity constraints preclude parallelized inference. As such, the implementation from [1] is serial. The contributions of this work are as follows. First, we demonstrate that effective parallelization is possible via a fast GPU implementation that scales gracefully with both the number of pixels and number of superpixels. Second, we show that the superpixels are improved by replacing the fixed and restricted spatial covariances from [1] with a flexible Bayesian prior. Quantitative evaluation on public benchmarks shows the proposed method outperforms the state-of-the-art. We make our implementation publicly available.

Index Terms— Superpixel, probabilistic model, inference, speedup, connectivity, parallelization

1. INTRODUCTION

A *Superpixel* is a coherent image patch, well aligned with image edges, such that its shape is not too irregular (see Fig. 1). While earlier usage of the term refers to polygonal patches [2], we adopt the more common usage of Ren and Malik [3]. As a compact intermediate representation, superpixel segmentation is often used as a pre-processing step in many image and vision analysis tasks [3, 4, 5, 6, 7, 8, 9, 10, 11, 12] enabling fast computation, lower-memory requirements and robustness. Temporal Superpixels (TSP) [1] are based on multiple single-frame superpixel segmentations: superpixels in one video frame, combined with a motion model, serve as a prior for the next frame. TSPs utilize a spatio-intensity Gaussian Mixture Model (GMM) that differs from traditional spatio-intensity GMMs [13, 14, 15] or K-means models [16] in that it imposes *connectivity constraints* that ensure every superpixel is simply connected. This eliminates the need for commonly-used post-processing connectivity heuristics, e.g., [17, 16]. We refer to this model as CC-GMM.

This research was partially supported by the Office of Naval Research Multidisciplinary Research Initiative (MURI) program, award N000141110688 and the Defense Advanced Research Projects Agency, award FA8650-11-1-7154.



Fig. 1: Example results using the proposed superpixel method

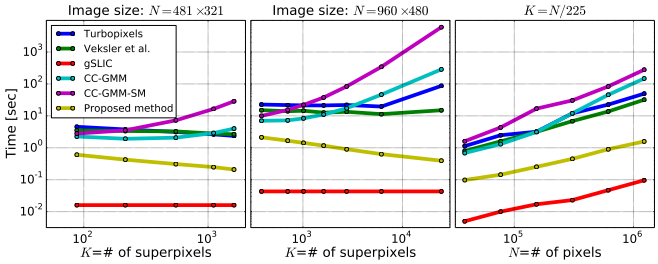


Fig. 2: Timing comparisons with other methods (both the abscissa and ordinate are scaled logarithmically). After gSLIC, the proposed method is the fastest. Quantitatively, however, the proposed method outperforms all methods; see Sec. 4.

Here, we improve upon CC-GMM in two ways. First, despite the connectivity, we show that inference may be parallelized, thus allowing a fast GPU implementation (see Fig. 2) that scales gracefully with an increase in the number of pixels, N, and whose computation time *decreases* as the number of superpixels, K, increases. Second, we replace the assumption on fixed spatial covariances with a Bayesian prior. While the subsequent development focuses on single frames, the improvements generalize to the TSP pipeline.

Both the latent parameters of the GMM and latent *labels* (i.e., the pixel-to-superpixel associations) must be inferred. Due to the connectivity, the labels are neither (statistically) independent nor conditionally independent which seemingly prohibits parallel inference. Thus, while the algorithm in [1] yields good results, it is serial (and hence slow), hindering its applicability, especially for multiple large images. Here we show that label updates can be efficiently computed in parallel by exploiting the fact that certain large *subsets* of the labels

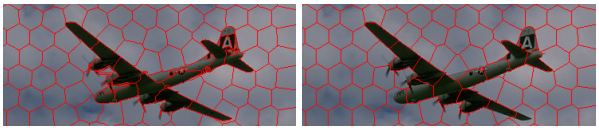


Fig. 3: The role of $\sigma_{\boldsymbol{f}}$. Left: $\sigma_{\boldsymbol{f}} = 10$. Right: $\sigma_{\boldsymbol{f}} = 25$.

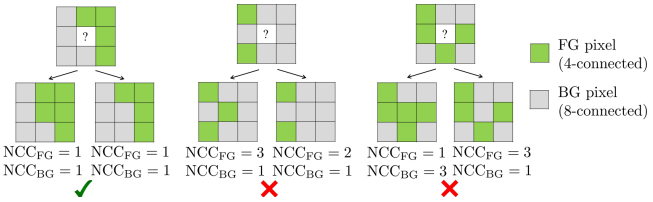


Fig. 4: Simple-point test for the binary case: 3 (out of 2^8) configurations. Left: this point is simple as *both* $\mathrm{NCC}_{\mathrm{BG}}$ and $\mathrm{NCC}_{\mathrm{FG}}$ are stable under label-change. Middle/right: Non-simple points. For technical reasons[24], 8-connectivity is used for BG while 4-connectivity is used for FG.

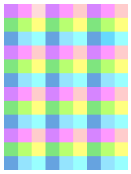


Figure 5: In this example, a 12×9 image is partitioned into 9 (colored) sets. This partition should not be confused with a superpixel segmentation. Let S be such a set. No 3×3 block contains more than a single element of S.

are conditionally independent. In contrast with [1], we also parallelize the parameter-estimation step. In [1] and [16], a single isotropic covariance, assumed to be known, is shared by all GMM components. We obtain improved results by relaxing this assumption via a Bayesian prior model.

Two other methods that ensure connectivity are the geometric-flow-based Turbopixels [18] and Veksler et al. [19] who use several ideas from [18] within an energy-optimization setting, specializing graph-cuts to superpixels in a manner that improves upon the N-cut method of [20]. The proposed method is faster computationally with better quantitative results than either method. Our speed comparison is based on the authors' *serial* implementations. It is unclear whether the method from [18] is parallelizable. The authors of [19] state their method is parallelizable; however, we are unaware of an existing implementation. Additionally, [20], [18] and [19] adopt an optimization framework in contrast to the generative probabilistic model of the proposed method.

The proposed method is faster than publicly-released implementations of other methods that explicitly model simple connectivity [18, 19, 1]. As compared to the GPU version of SLIC [16, 21] it is slower, though exhibits superior performance on benchmarks. This is a consequence of the more restrictive K-means assumption of SLIC combined with a separate post-processing connectivity heuristic as compared with the GMM integrated with explicit connectivity constraints of the proposed method. The latter induces additional computations. Quantitative results, obtained using publicly-available benchmarks [22, 23], show the proposed method outperforms all the methods above (SLIC and CC-GMM included). Similar to [13, 16, 1], we have two user-defined parameters: one controls the relative weights of the location and color; the other (together with N) determines K.

2. MODEL

Let $\boldsymbol{x}_i = (\boldsymbol{l}_i, \boldsymbol{f}_i)$ be the observation associated with the ith pixel, where $\boldsymbol{l}_i$ is the 2D location and $\boldsymbol{f}_i$ is a d-dimensional feature, e.g., if $\boldsymbol{f}_i$ is the color $d = 3$. As in [16] and [1], we use the *Lab* color space to facilitate a fair comparison, though extension to other features is straightforward. The latent label of the pixel is denoted by $z_i \in \{1, \ldots, K\}$. Our goal is to infer $\boldsymbol{z} \triangleq (z_1, \ldots, z_N) \in \mathcal{Z} \triangleq \{1, \ldots, K\}^N$ conditioned on observations $\boldsymbol{X} \triangleq (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N)$ utilizing the following model. The $\{\boldsymbol{x}_i\}_{i=1}^N$ are modeled as identically-distributed, but *not* independent, samples from a GMM: $\boldsymbol{x}_i \sim p(\boldsymbol{x}_i; \theta) = \sum_{j=1}^K w_j p(\boldsymbol{x}_i | z_i = j; \theta_j)$ where $\theta_j = (\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$ are the parameters of the jth Gaussian (i.e., $p(\boldsymbol{x}_i | z_i = j; \theta_j) = \mathcal{N}(\boldsymbol{x}_i; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$), $w_j \triangleq \Pr(z_i = j)$, and $\theta = (\theta_1, \ldots, \theta_K, w_1, \ldots, w_K)$. Particularly,

$$\boldsymbol{\mu}_j = \begin{bmatrix} \boldsymbol{\mu}_j^{\boldsymbol{f}} \\ \boldsymbol{\mu}_j^{\boldsymbol{l}} \end{bmatrix} \quad \boldsymbol{\Sigma}_j = \begin{bmatrix} \boldsymbol{\Sigma}_j^{\boldsymbol{f}} & \mathbf{0}_{2\times d} \\ \mathbf{0}_{d\times 2} & \boldsymbol{\Sigma}_j^{\boldsymbol{l}} \end{bmatrix} , \tag{1}$$

$$p(\boldsymbol{x}_i | z_i = j; \theta_j) = \mathcal{N}(\boldsymbol{l}_i; \boldsymbol{\mu}_j^{\boldsymbol{l}}, \boldsymbol{\Sigma}_j^{\boldsymbol{l}}) \mathcal{N}(\boldsymbol{f}_i; \boldsymbol{\mu}_j^{\boldsymbol{f}}, \boldsymbol{\Sigma}_j^{\boldsymbol{f}}) . \tag{2}$$

As is common in spatio-intensity models [13, 14, 15, 16, 1], $\boldsymbol{l}_i$ and $\boldsymbol{f}_i$ are independent conditioned on z_i; i.e, $\boldsymbol{\Sigma}_j$ is block diagonal. As in [13] and [1], we let $\boldsymbol{\Sigma}_j^{\boldsymbol{f}} = \sigma_{\boldsymbol{f}}^2 \boldsymbol{I}_{d\times d}$, where the user-defined $\sigma_{\boldsymbol{f}}$ determines the weights of $\boldsymbol{x}_i$ and $\boldsymbol{l}_i$ (Fig. 3). In [1] and [16] the $\boldsymbol{\Sigma}_j^{\boldsymbol{l}}$'s are assumed to be known, isotropic, and identical. In contrast, here the $\boldsymbol{\Sigma}_j^{\boldsymbol{l}}$'s are *latent*, different from each other, and need to be neither diagonal nor isotropic. Specifically, $\boldsymbol{\Sigma}_j^{\boldsymbol{l}} \sim \mathcal{W}^{-1}(\boldsymbol{\Psi}, \nu)$ where $\mathcal{W}^{-1}$ is an Inverse-Wishart (IW) prior, ν are the *pseudo-counts*, and $\boldsymbol{\Psi}$ is symmetric and positive-definite. If $\boldsymbol{\Psi}$ is diagonal and isotropic, which does not imply $\boldsymbol{\Sigma}_j^{\boldsymbol{l}}$ is such, then as $\nu \to \infty$ this generalization coincides with [1]. As in [1], the $\boldsymbol{x}_i$'s are dependent due to our use of the following probability distribution: all elements of $\mathcal{Z}$ that consist solely of simply-connected superpixels are equiprobable; all other elements have zero probability. As a result, inferred superpixels satisfy the constraint that, for every j, the set $\{\boldsymbol{l}_i : z_i = j\}$ is simply connected.

3. INFERENCE

Iterative hard-assignment inference methods in a GMM include sampling and hard Expectation-Maximization (EM). Both approaches alternate between label updates ('E step') and parameter updates ('M step'), and both can be used for CC-GMM; however, special care must be taken in the E step.

For concreteness, we here focus on the hard EM approach; the sampling approach can be treated similarly.

Initialization: A valid initial segmentation is any nonzero probability element of $\mathcal{Z}$. Specifically, we use a hexagonal tiling, where the area of each hexagon, A, is user-specified. The area, along with N, implies K: $K \approx N/A$. We let $\nu = cA$ and $\mathbf{\Psi} = A^2 I_{2\times 2}$ (so $\frac{\mathbf{\Psi}}{\nu} = \frac{A}{c} I_{2\times 2}$). The higher ν is, the more influential $\mathbf{\Psi}$ is. We set c to 5 (the results for c in the range $[1, 10]$ being similar).

M Step (similar to the M step in GMMs with an IW prior): Given $\boldsymbol{z}$, for each $j \in \{1, \ldots, K\}$, compute the *sufficient statistics*, $(\boldsymbol{f}'_j, \boldsymbol{l}'_j, \boldsymbol{l}''_j) \triangleq \sum_{i:z_i=j} (\boldsymbol{f}_i, \boldsymbol{l}_i, \boldsymbol{l}_i \boldsymbol{l}_i^T)$ and $n_j \triangleq \sum_{i=1}^{N} \delta_{j,z_i}$ ($\delta_{\cdot,\cdot}$ is the Kronecker delta). Then, obtain ML estimates for $(w_j, \boldsymbol{\mu}^{\boldsymbol{l}}_j, \boldsymbol{\mu}^{\boldsymbol{f}}_j)$, and a posterior mean[1] for $\mathbf{\Sigma}^{\boldsymbol{l}}_j$:

$$(\widehat{w_j}, \widehat{\boldsymbol{\mu}}^{\boldsymbol{f}}_j, \widehat{\boldsymbol{\mu}}^{\boldsymbol{l}}_j) = (\frac{n_j}{N}, \frac{\boldsymbol{f}'_j}{n_j}, \frac{\boldsymbol{l}'_j}{n_j}), \quad \widehat{\mathbf{\Sigma}}^{\boldsymbol{l}}_j = \frac{\boldsymbol{l}''_j - \frac{\boldsymbol{l}'_j \boldsymbol{l}'^T_j}{n_j} + \mathbf{\Psi}}{n_j + \nu - 3} \quad (3)$$

(by conjugacy[2], $p(\mathbf{\Sigma}^{\boldsymbol{l}}_j | \boldsymbol{X}, \boldsymbol{z}) = \mathcal{W}^{-1}(\boldsymbol{l}''_j - \frac{\boldsymbol{l}'_j \boldsymbol{l}'^T_j}{n_j} + \mathbf{\Psi}, \nu + n_j)$). Due to space limits, and as parallel EM implementations for standard GMMs exist [26], we omit the implementation details of this step. However, the following design choices, made because K is typically large, are worth mentioning. First, our implementation parallelizes this step over superpixels. Thus, as K *increases*, our running-time *decreases* since, per superpixel, there are fewer summands. This behavior is in contrast with some superpixel implementations, including [1]. That said, if a small K (say, $K < 50$) is desired, parallelizing over pixels may be better. Second, rather than using reductions for the sums, we opted to considerably simplify the code and bookkeeping by using *atomicAdd*. While this means that, within a superpixel, the n_j summands are summed serially, it is still very fast since n_j is small (since K is large). Particularly, as it is fast enough to make the E step the computational bottleneck, we avoid further optimizations of the M step.

E Step (this step is affected by the connectivity): Assuming each of the current superpixels is simply connected, only connectivity-preserving label changes are considered. Clearly, only pixels on inter-superpixel boundaries may change their labels. Even for these, however, connectivity tests must be made (to avoid splits or topological "handles"). In the binary case ($K = 2$), this is done as follows [24]. One considers the binary labels in a 3×3 block about the pixel of interest and counts the Number of Connected Components (NCC) for both labels in two conditions: when the central label is 0 ('BG') and when it is 1 ('FG'). A point is called *simple* if changing its label changes the NCC of neither label; see Fig. 4 for examples. A binary-label change preserves connectivity if and only if the point is simple [24], which is tested efficiently via table lookup[3]. If $K > 2$, label j is a possible assignment if and only if the point is simple w.r.t. it[4]. Let $\eta_4(i)$ and $\eta_8(i)$ denote the 4- and 8-connectivity neighbors of pixel i, let $\boldsymbol{z}|_{\eta_4(i)}$ and $\boldsymbol{z}|_{\eta_8(i)}$ denote their corresponding *sets* of labels, and let $\mathrm{SP}(\boldsymbol{z}_{\eta_8(i)}) \subset \boldsymbol{z}|_{\eta_4(i)}$ denote the labels w.r.t. which pixel i is a simple point. The label update rule is:

$$\text{If } \mathrm{SP}(\boldsymbol{z}_{\eta_8(i)}) = \emptyset \text{ then avoid update; otherwise:}$$
$$\widehat{z}_i = \underset{j \in \mathrm{SP}(\boldsymbol{z}_{\eta_8(i)})}{\arg\max} \; \widehat{w}_j \mathcal{N}(\boldsymbol{f}_i; \widehat{\mu}^j_{\boldsymbol{f}}, \Sigma^j_{\boldsymbol{f}}) \mathcal{N}(\boldsymbol{l}_i; \widehat{\mu}^j_{\boldsymbol{l}}, \widehat{\Sigma}^j_{\boldsymbol{l}}) \,. \quad (4)$$

Modulo our use of $\widehat{\mathbf{\Sigma}}^{\boldsymbol{l}}_j$, this is similar to [1]. The problem, however, is that *updating the labels of two adjacent pixels in parallel can break connectivity*. Equivalently, $p(\boldsymbol{z}, \boldsymbol{X}|\theta) \neq \prod_{i=1}^{N} p(z_i, \boldsymbol{X}|, \theta)$ and $p(\boldsymbol{z}|\boldsymbol{X}, \theta) \neq \prod_{i=1}^{N} p(z_i|\boldsymbol{X}, \theta)$; i.e., the labels are not conditionally independent.

Consequently, label updates cannot be done in parallel leading to the serial implementation of [1]. We note, however, that if S is a pixel set such that no 3×3 block contains more than one element of S, then the labels of the pixels in S *are* conditionally independent (see Fig. 5). It follows that Eqn. (4) may be applied in parallel to all pixels in S without breaking connectivity. Thus, while we cannot parallelize the E step over all N pixels, we can parallelize it over $N/9$ pixels at the time. The procedure is summarized in Algorithm 1.

```
for m ∈ {1, ..., N_iter} do
    for j ∈ {1, ..., K} do in parallel
        Apply Eqn. (3). // M step
    for q ∈ {1, ..., M_iter} do
        for row ∈ {0, 1, 2} do
            for col ∈ {0, 1, 2} do
                B = {x_i : z_i ≠ z_i' for some i' ∈ η_4(i)}
                S = {x_i : mod (l_i, 3) = (col, row)}
                for x_i ∈ S ∩ B do in parallel
                    Apply Eqn. (4). // E step
```

Algorithm 1: We use $N_{\mathrm{iter}} = \sqrt{A}$ and $M_{\mathrm{iter}} = 10$ which, empirically, sufficed for convergence. Note that membership decisions (for the sets B,S and $B \cap S$) are also done in parallel over all N pixels.

4. RESULTS

We used two benchmarks: BSDS500 [22] and Chemnitz' optical-flow-based superpixel benchmark toolbox [27, 23]. For BSDS500, we report standard metrics: Under Segmentation Error (USE) and Boundary Recall (BR). The Chemnitz toolbox computes two evaluation metrics on the MPI Sintel dataset [28]: Motion Under Segmentation Error (MUSE) and

[1] For 2×2 matrices, the difference between the mean and the mode of an IW distribution is small, and thus both choices lead to similar results.

[2] For more details on conjugate priors, see [25].

[3] The test uses 8 binary labels: all labels in the block but the central one. Each of the 2^8 configurations corresponds to a row in a boolean table. The table is then accessed using 1-byte encoding of the current 8 labels.

[4] I.e., label j is viewed as FG while all others are viewed as BG.

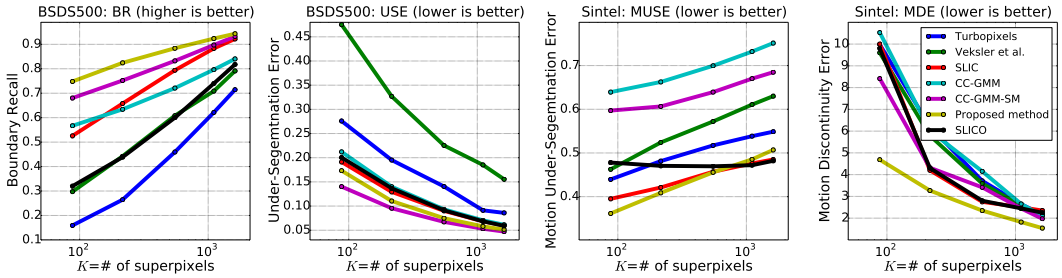
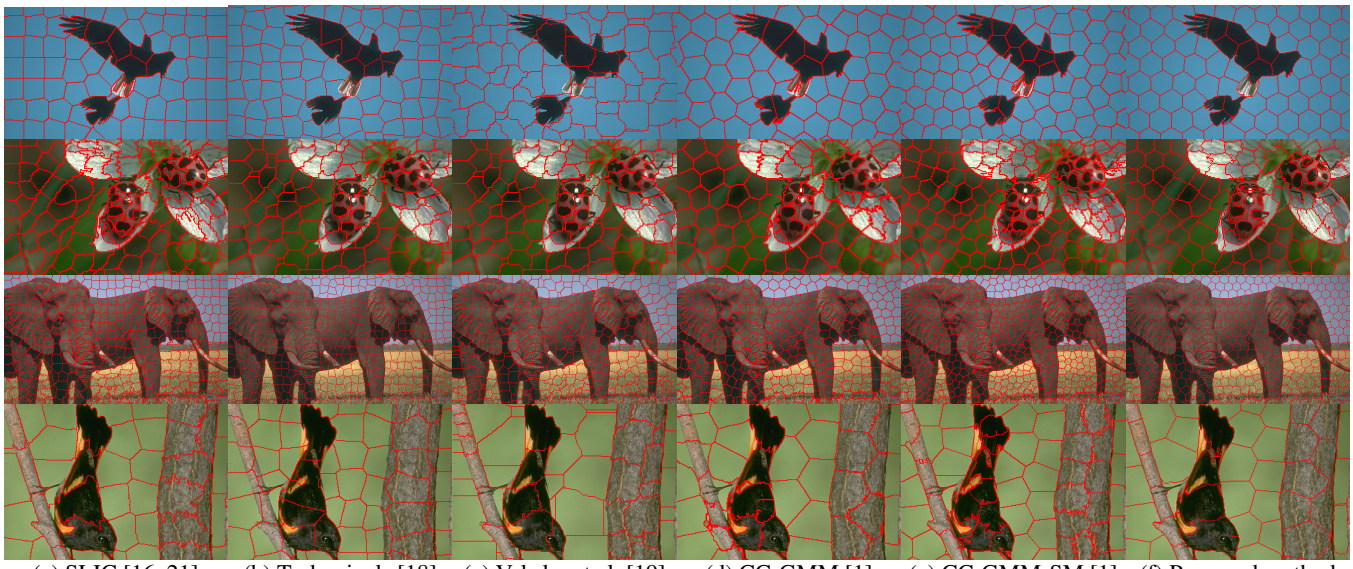


Fig. 6: Quantitative results: the abscissa is scaled logarithmically. The proposed method performs best excepting the second plot. There, CC-GMM slightly outperforms due to computationally expensive (see Fig. 2) split-and-merge moves. Interestingly, via the improved spatial-covariance modeling, the proposed method outperforms CC-GMM-SM on the other 3 indices.



(a) SLIC [16, 21] (b) Turbopixels [18] (c) Veksler et al. [19] (d) CC-GMM [1] (e) CC-GMM-SM [1] (f) Proposed method

Fig. 7: Select results. Additional examples, in higher resolution, are available at:
`http://groups.csail.mit.edu/vision/sli/projects/fastSCSP/visual.pdf`

Motion Discontinuity Error (MDE). The methods we compared with (using implementations from the authors' websites) are Veksler et al., [19], Turbopixels [18], SLIC [16], SLICO [16] and two variants from [1]. In the first, CC-GMM, K is fixed. In the second, CC-GMM-SM, K may change using several split-and-merge moves. SLIC has a user-defined parameter similar to our $\sigma_{\boldsymbol{f}}$. For Fig. 6 we used $\sigma_{\boldsymbol{f}} = 5$, the optimal value for both our method and SLIC. SLICO is a variant of SLIC in which the parameter is decided automatically. For timing SLIC, we used the faster gSLIC [21]. We used NVIDIA GeForce GTX 780 for the GPU implementations and Intel(R) Xeon(R) E5-2670 v3 @ 2.30GHz for the serial ones. From Fig. 6, our method is superior on standard benchmarks. Timings (Fig. 2) show our method is the fastest (following gSLIC). Fig. 7 provides a visual comparison.

5. CONCLUSION

We have shown how to parallelize CC-GMM inference and proposed a fast method for high-quality simply-connected superpixels. The proposed parallelization approach is straightforward and provides substantial speedups. The ideas presented may lead to even faster methods. We also demonstrated that Bayesian estimates of spatial covariances improve the resulting superpixels. Qualitative evaluation shows the method outperforms state-of-the-art methods on a variety of benchmarks. Interestingly, the improved spatial-covariance modeling yields results that, overall, obviate the need for expensive splits/merges. An interesting research direction is the incorporation of Bayesian hierarchical models (which are more flexible and easier to parallelize than splits/merges) within the proposed method. Finally, *incremental* updates of the sufficient statistics are likely to improve the speed, especially when K is small. Our implementation is available at `https://github.com/freifeld/fastSCSP`

6. REFERENCES

[1] Jason Chang, Donglai Wei, and John W Fisher III, "A video representation using temporal superpixels," in *IEEE CVPR*, 2013.

[2] Uwe Rauschenbach, René Rosenbaum, and Heidrun Schumann, "A flexible polygon representation of multiple overlapping regions of interest for wavelet-based image coding," in *ICIP*, 2001.

[3] Xiaofeng Ren and Jitendra Malik, "Learning a classification model for segmentation," in *IEEE ICCV*, 2003.

[4] Derek Hoiem, Alexei A Efros, and Martial Hebert, "Geometric context from a single image," in *ICCV*. IEEE, 2005.

[5] Xuming He, Richard S Zemel, and Debajyoti Ray, "Learning and incorporating top-down cues in image segmentation," in *ECCV*. Springer, 2006.

[6] Caroline Pantofaru, Cordelia Schmid, and Martial Hebert, "Object recognition by integrating multiple image segmentations," in *ECCV*. Springer, 2008.

[7] Brian Fulkerson, Andrea Vedaldi, and Stefano Soatto, "Class segmentation and object localization with superpixel neighborhoods," in *IEEE ICCV*, 2009.

[8] Sylvain Boltz, Frank Nielsen, and Stefano Soatto, "Earth mover distance on superpixels," in *ICIP*. IEEE, 2010.

[9] Joseph Tighe and Svetlana Lazebnik, "Superparsing: scalable nonparametric image parsing with superpixels," in *ECCV*. Springer, 2010.

[10] Soumya Ghosh and Erik B Sudderth, "Nonparametric learning for layered segmentation of natural images," in *IEEE CVPR*, 2012.

[11] Zhenguo Li, Xiao-Ming Wu, and Shih-Fu Chang, "Segmentation using superpixels: A bipartite graph partitioning approach," in *CVPR*. IEEE, 2012.

[12] Shaul Oron, Aharon Bar-Hillel, Dan Levi, and Shai Avidan, "Locally orderless tracking," in *CVPR*. IEEE, 2012.

[13] Chad Carson, Serge Belongie, Hayit Greenspan, and Jitendra Malik, "Blobworld: Image segmentation using expectation-maximization and its application to image querying," *IEEE Tran. on PAMI*, 2002.

[14] Hayit Greenspan, Amit Ruf, and Jacob Goldberger, "Constrained gaussian mixture model framework for automatic segmentation of MR brain images," *IEEE Tran. on MI*, 2006.

[15] Oren Freifeld, Hayit Greenspan, and Jacob Goldberger, "Multiple sclerosis lesion detection using constrained GMM and curve evolution," *International Journal of Biomedical Imaging*, 2009.

[16] Radhakrishna Achanta, Appu Shaji, Kevin Smith, Aurelien Lucchi, Pascal Fua, and Sabine Susstrunk, "SLIC superpixels compared to state-of-the-art superpixel methods," *IEEE Tran. on PAMI*, 2012.

[17] Pedro F Felzenszwalb and Daniel P Huttenlocher, "Efficient graph-based image segmentation," *IJCV*, 2004.

[18] Alex Levinshtein, Adrian Stere, Kiriakos N Kutulakos, David J Fleet, Sven J Dickinson, and Kaleem Siddiqi, "Turbopixels: Fast superpixels using geometric flows," *IEEE Tran. on PAMI*, 2009.

[19] Olga Veksler, Yuri Boykov, and Paria Mehrani, "Superpixels and supervoxels in an energy optimization framework," in *ECCV*. Springer, 2010.

[20] Jianbo Shi and Jitendra Malik, "Normalized cuts and image segmentation," *IEEE Tran. on PAMI*, 2000.

[21] Carl Yuheng Ren and Ian Reid, "gslic: a real-time implementation of SLIC superpixel segmentation," *University of Oxford, Department of Engineering, TR*, 2011.

[22] Pablo Arbelaez, Michael Maire, Charless Fowlkes, and Jitendra Malik, "Contour detection and hierarchical image segmentation," *IEEE Tran. on PAMI*, 2011.

[23] Peer Neubert and Peter Protzel, "Evaluating superpixels in video: Metrics beyond figure-ground segmentation," in *BMVC*, 2013.

[24] Giles Bertrand, "Simple points, topological numbers and geodesic neighborhoods in cubic grids," *Pattern recognition letters*, 1994.

[25] Andrew Gelman, John B Carlin, Hal S Stern, David B Dunson, Aki Vehtari, and Donald B Rubin, *Bayesian data analysis*, CRC press, 2013.

[26] NSLP Kumar, Sanjiv Satoor, and Ian Buck, "Fast parallel expectation maximization for Gaussian mixture models on GPUs using CUDA," in *High Performance Computing and Communications*, 2009.

[27] Peer Neubert and Peter Protzel, "Superpixel benchmark and comparison," in *Proc. Forum Bildverarbeitung*, 2012.

[28] Daniel J Butler, Jonas Wulff, Garrett B Stanley, and Michael J Black, "A naturalistic open source movie for optical flow evaluation," in *ECCV*. Springer, 2012.